

FORMATION IDU

Bonnes Pratiques du Milieu Professionnel Data

Code propre · Travail en équipe · Communication · Éthique · RGPD
Tout ce que l'école n'enseigne pas mais que les recruteurs attendent

Ce que les recruteurs cherchent au-delà des compétences techniques

Avoir un bon niveau technique est nécessaire mais pas suffisant. Les entreprises recrutent des personnes avec qui elles vont travailler pendant des années. Elles veulent des profils qui **écrivent du code maintenable, communiquent clairement, respectent les données personnelles, et contribuent positivement à l'équipe**. Ce guide couvre exactement ces aspects.

1. Écrire du code professionnel et maintenable

Le code que tu écris sera lu, modifié et maintenu par d'autres personnes. Un code professionnel n'est pas forcément complexe — il est **lisible, testé et documenté**.

PEP 8 — Style Python

Nommage clair (snake_case), lignes < 88 caractères, imports organisés. Utilise Black (formateur auto) et flake8 (linter).

→ Ne jamais nommer une variable 'x' ou 'df2' dans du code de production. Préfère 'monthly_sales_by_region' à 'df_msb'.

Typage statique (type hints)

Annoter les fonctions avec les types Python (-> dict, : str, etc.). Utiliser mypy pour validation statique. Standard dans les équipes sérieuses.

→ `def compute_churn_rate(customers: pd.DataFrame, period: str) -> float:`

Docstrings

Chaque fonction publique doit avoir une docstring (NumPy ou Google style). Décrit ce que la fonction fait, ses paramètres et ce qu'elle retourne.

→ Sphinx peut auto-générer de la documentation à partir des docstrings.

Tests unitaires

pytest est le standard. Vise 70%+ de couverture sur le code critique. Teste les cas limites, pas seulement le cas nominal.

→ Les tests sont aussi une documentation. Ils montrent comment le code doit s'utiliser.

Gestion des erreurs

Toujours utiliser try/except explicite. Logger les erreurs avec logging (pas print). Lever des exceptions personnalisées avec messages clairs.

→ Un pipeline qui tombe silencieusement est pire qu'un pipeline qui lève une erreur propre.

Configuration & secrets

Ne jamais hardcoder des credentials dans le code. Utiliser .env + python-dotenv, ou des secrets managers (AWS Secrets Manager, Vault).

→ Le fichier .env doit toujours être dans .gitignore. Sans exception.

2. Travail en équipe avec Git

Git est l'outil de collaboration numéro un en développement. Maîtriser les workflows Git est indispensable pour travailler en équipe sans créer de chaos.

Pratique Git	Ce qu'on fait	Pourquoi c'est important
Feature Branches	Une branche par feature/bug (feature/add-spark-job, fix/resolve-bug)	Isolation du travail, code review facilitée
Commits atomiques	Un commit = un changement logique. Message clair : feat: stocker les données	Historique lisible, revert facile si bug
Pull Requests (PR)	Code review par un collègue avant merge. Toujours.	Partage de connaissance, détection de bugs
Conventional Commits	feat:, fix:, docs:, refactor:, test: en préfixe de commit	Changelog automatique, lisibilité
Protection de main/master	Interdire les push directs sur la branche principale	Protège la branche de production
.gitignore rigoureux	Ne pas committer .env, __pycache__, data/, modèles ML	Sécurité, légèreté du repo
Tagging des releases	git tag v1.0.0 à chaque version stable déployée	Traçabilité, rollback possible

3. Documentation — L'art souvent négligé

Un bon Data Engineer documente son travail. La documentation ne sert pas qu'aux autres : elle te servira à toi dans 6 mois quand tu reviendras sur un projet oublié.

■ README du projet	■ Outils : Markdown · Badges (shields.io) · GIFs de démo
Description, prérequis, installation, utilisation, exemples. Le README est la carte de visite du projet.	
■ Architecture Decision Records (ADR)	■ Outils : Template ADR Markdown · adr-tools
Documente POURQUOI tu as fait tel choix technique. Exemple : 'Pourquoi Kafka plutôt que RabbitMQ pour ce projet ?'	
■ Data Dictionary	■ Outils : Notion, Confluence, ou fichier YAML
Catalogue des datasets : signification de chaque colonne, source, type, valeurs possibles, fréquence de mise à jour.	
■ Diagrammes d'architecture	■ Outils : draw.io, Mermaid.js, Lucidchart
Architecture générale du pipeline (source → ingestion → stockage → transformation → exposition). Mermaid.js dans GitHub.	
■ Runbooks opérationnels	■ Outils : Confluence, Notion, ou wiki GitHub
Procédures en cas d'incident : 'Si le job Airflow échoue, voici comment diagnostiquer et relancer'.	

4. RGPD, Protection des données & Éthique IA

Le Règlement Général sur la Protection des Données (RGPD) est en vigueur depuis 2018. Tout professionnel de la data en France doit le connaître. Les violations de données peuvent entraîner des amendes jusqu'à **4% du chiffre d'affaires mondial** de l'entreprise. C'est aussi un critère de recrutement : les entreprises veulent des profils conscients de ces enjeux.

■ ■ Données personnelles

Toute information permettant d'identifier une personne : nom, email, adresse IP, données de géolocalisation, cookie d'identification.

À retenir : Principe de minimisation : ne collecter QUE les données nécessaires.

■ ■ Base légale du traitement

Consentement, contrat, obligation légale, intérêt légitime. Chaque traitement de données doit avoir une base légale identifiée.

À retenir : Documenter la base légale dans le Registre des Activités de Traitement (RAT).

■ ■ Droit à l'effacement (droit à l'oubli)

Toute personne peut demander la suppression de ses données. Ton pipeline doit pouvoir supprimer les données d'un individu en cascade.

À retenir : Concevoir les pipelines avec le 'Right to be Forgotten' dès le départ.

■ ■ Pseudonymisation & Chiffrement

Pseudonymiser les données avant traitement. Chiffrer les données sensibles au repos (AES-256) et en transit (TLS).

À retenir : Ne jamais stocker des données sensibles en clair, même en dev.

■ ■ Privacy by Design

Intégrer la protection des données dès la conception du système, pas en ajout final.

À retenir : Principe fondamental du RGPD — s'applique à toute architecture data.

■ ■ IA éthique & biais algorithmiques

Les modèles ML peuvent amplifier des biais existants (genre, ethnie, âge). Obligation de détecter et corriger ces biais.

À retenir : Fairlearn, AI Fairness 360 — outils de détection des biais dans les modèles.

5. Communication dans les équipes data

Un bon Data Engineer ou Data Scientist doit pouvoir expliquer son travail technique à des non-techniciens (business, direction) et collaborer avec des collègues de disciplines différentes.

■ Data storytelling

- Une visualisation doit raconter une histoire, pas juste afficher des chiffres.
- Toujours commencer par le 'So what?' : quelle est la décision que cette analyse aide à prendre ?
- Adapter le niveau de détail technique à son audience (CTO ≠ business analyst ≠ DG).

■ Écrire des rapports d'analyse

- Structure : Contexte → Méthodologie → Résultats → Recommandations → Limites.
- Mettre les conclusions en premier (executive summary d'1 page max).
- Quantifier les impacts : 'Cette optimisation réduit le temps de traitement de 40%'.

■■ Présentations techniques

- Limiter les slides de code. Préférer des diagrammes et des métriques clés.
- Toujours préparer une version courte (5 min) et une version longue (20 min) du même sujet.
- La règle du 'et alors ?' : chaque slide doit répondre à cette question.

■ Travail en équipe Agile/Scrum

- Daily standup : ce que j'ai fait / ce que je fais / mes blocages. Pas plus de 2 minutes.
- Sprint planning : estimer les tâches en story points. Être honnête sur les estimations.
- Rétrospectives : espace de feedback bienveillant, pas de reproches personnels.

■ Donner et recevoir du feedback

- Feedback sur le code (code review) : se concentrer sur le code, pas sur la personne.
- Exemple : 'Cette fonction fait trop de choses, je suggère de la séparer en deux' (OK).
- 'Ton code est illisible' (jamais acceptable dans un contexte professionnel).

6. La mentalité du professionnel de la data

■ Ownership — S'approprier son travail

Un vrai professionnel ne dit pas 'ça tombe en dehors de mon périmètre'. Si ton pipeline impacte le dashboard business, c'est TON problème aussi. Les meilleurs profils pensent 'end-to-end' et prennent responsabilité du résultat final.

■ Fail fast, learn faster

Dans un environnement Agile, les erreurs sont des apprentissages. Ne pas cacher ses erreurs. Les documenter, les partager en rétrospective, et mettre en place des garde-fous pour éviter qu'elles se reproduisent.

■ Sécurité dès la conception

Principe du moindre privilège : un service doit avoir accès uniquement aux ressources dont il a strictement besoin. Auditer les accès régulièrement. Jamais de credentials dans le code ou les logs.

■ Apprendre en continu — Le secteur évolue vite

Python 3.12, Spark 4.0, Delta Lake 3.0... Les outils changent en permanence. Les meilleurs professionnels réservent du temps chaque semaine pour se former : blogs techniques (Towards Data Science), papiers de recherche, conférences (Data+AI Summit).

■ Penser au coût et à la performance

En entreprise, chaque requête BigQuery coûte de l'argent. Chaque job Spark consomme des ressources cloud. Les bons Data Engineers optimisent leurs pipelines pour la performance ET le coût.

■ Collaborer avec les métiers

Le meilleur pipeline du monde ne sert à rien si les business analysts ne comprennent pas comment l'utiliser. Investis du temps à comprendre les besoins réels avant de coder.

Checklist avant un entretien technique

Cette checklist couvre ce que les recruteurs testent en entretien :

- Savoir expliquer un projet de ton portfolio sans regarder tes notes
- SQL : écrire une requête avec JOIN, GROUP BY, HAVING, sous-requêtes, fenêtrage (OVER PARTITION BY)
- Python : lire et nettoyer un CSV avec Pandas en live coding
- Expliquer la différence entre Data Lake, Data Warehouse et Data Lakehouse
- Décrire un pipeline ETL que tu as construit (sources → transformations → destination)
- Savoir parler de RGPD : ce qu'est une donnée personnelle, le droit à l'effacement
- Expliquer ce qu'est le partitionnement dans Spark et pourquoi c'est important
- Avoir des questions intelligentes à poser au recruteur (outils utilisés, stack data)
- GitHub à jour avec au moins 3 projets avec README propres
- LinkedIn à jour avec les compétences techniques et les projets

Sources : CNIL.fr · RGPD.fr · Atlassian Agile Guide · Google Engineering Practices · Martin Fowler (Clean Code) · Référentiel IDU RNCP35994.